Comparison: Array vs Linked List in C#

Feature	Array	Linked List
Memory Allocation	Contiguous block in memory	Nodes allocated separately (non-contiguous)
Access Time	Fast random access (O(1) via index)	Sequential access (O(n) to reach a node)
Insertion/Deletion	Expensive in middle (O(n))	Efficient in middle (O(1) if node known)
Size	Fixed size (unless using dynamic arrays like List<T>)	Dynamic size (grows/shrinks easily)
Memory Usage	Less memory overhead	More memory (extra pointer per node)
Cache Performance	Better (due to locality)	Poorer (nodes scattered in memory)
Built-in Types	Array, List<T>	LinkedList<T>

When to Use:

- Array: When you need fast access by index and a fixed or predictable size.
- Linked List: When you need frequent insertions/deletions and don't care about direct indexing.